

Core Platform Framework

01 개발자 매뉴얼

업무 개발자 · 서비스 개발자 · 기술 리드

문서 목적	온라인·연계 업무를 만들 때 CPF Public API, Paging, Transaction, Idempotency, Messaging, 외부연계, Security 와 Test/인계를 실제로 적용하는 정보입니다.
Repository / Branch	freeangelsun/202412_01_CPF / master
기준 Source / 정보	Repository f6d7080c5a14b7dd7595093f9497470169e18d80 Product Source f0aa49f29cba3cfd6ae12b0ddd4e118d05fff16c 최상위 요구사항 cpf-docs/governance/CPF_FINAL_TARGET_REQUIREMENTS.md 문서 표준 cpf-docs/specification/CPF_DOCUMENTATION_STANDARD.md
문서 성격	완성 기준 사용자 문서. 제품이 완료된 상태에서 사용자가 설치·개발·운영·복구·검 증 업무를 끝낼 수 있도록 정상 절차, 실패 판정, 복구, 확인 근거를 기술합니다.

읽는 방법 먼저 “빠른 찾기”에서 질문을 찾고, 상세 설명은 각 기능의 정보 절로 이동하십시오. 다른 매뉴얼은 같은 내용을 반복하지 않고 정보 절을 참조합니다.

빠른 찾기 - 질문에서 정보 절로

개발자가 묻는 질문	이 문서의 정보 위치	다른 문서 연결
공통 Utility 는 무엇이 있고 어떻게 쓰나?	§2 CPF Public Utility API Catalog	00 은 Navigator 만 제공
Paging Class/import/default/error 예제는?	§3 Paging Reference	기술사양서는 정책, 실제 사용법은 이 절
Remote HTTP 호출 Public API 는?	§4 CpfHttpClient	ServiceCallEngine 내부 구현은 직접 생성하지 않음
Local Transaction 으로 묶어도 되는가?	§6 Transaction 판단 흐름	도식 + 상황별 표
A→B→C 가 다른 인스턴스면 rollback/log 는?	§8 다중 Owner Transaction Lineage	운영 조치는 05 §4
응답 유실/timeout 이면 retry 해도 되나?	§9 Idempotency / UNKNOWN_RESULT	결과 확인 전 비역등 재호출 금지
Message/File/외부기관과 TX 는?	§10~§11	Outbox/Inbox, Reconcile/Compensation
운영 Log/FileLog 장애는 개발자가 직접 처리?	§13 개발자 Logging 책임	FileLog 내부 복구는 05 §5

1. 개발 기능 Reference - Public API / SPI / Framework Contract / Internal

구분	개발자가 해도 되는 것	하지 말아야 할 것	정보
Public API	업무 Source 에서 import/호출	내부 구현체 직접 new	이 문서 §2~§5
SPI	고객 Provider/Adapter 구현	Framework 내부 상태 직접 변경	해당 기능 절
Framework Contract	Header/Idempotency/Transaction/Trace 규칙 준수	Header/상태를 임의 의미로 재정의	§6~§13
Internal	Framework 가 자동 적용	CpfFileLogWriter/RecoverySpool 등 내부 Class 직접 생성	05 운영 매뉴얼

2. CPF Public Utility API Catalog

패키지 기준: `cpf-core/src/main/java/com/cpf/core/api/util/\*\*`. 아래 Class 는 현재 Product Source 에 실제 존재하는 공개 Utility 입니다. Utility 는 업무 의미를 숨기는 만능 Helper 가 아니라 반복되는 기술 입력 정규화·검증을 줄이는 용도로 사용됩니다.

Class	주요 Method	사용 목적	오류·경계
CpfStrings	hasText, trimOrNull, defaultIfBlank, requireText, abbreviate, left, right, joinNonBlank, normalizeCode, normalizeCodeOrNull, repeat	null/blank·코드 문자열 정규화	requireText/길이·count 조건 위반은 IllegalArgumentException
CpfDates	parse, parseBasic, parse(formatter), parseOrNull, format, formatBasic, betweenInclusive	LocalDate 파싱·형식화·범위	잘못된 날짜는 IllegalArgumentException; parseOrNull 은 null
CpfTimes	now, now(zone), today, nowDateTime, nowDateTimeMillis, toInstant, toLocalDateTime	Instant/LocalDateTime 변환·표준 문자열	ZonedId 누락 시 IllegalArgumentException
CpfDecimals	parse, scale, zeroOrNull, isZero	금액/Decimal scale·rounding 명시	잘못된 숫자는 IllegalArgumentException; roundingMode null 은 HALF_UP
CpfFiles	safeFileName, extension, resolveChild	파일명·Root 하위경로 안전화	경로 포함 filename / Root 탈출은 IllegalArgumentException
CpfHashes	sha256(String/byte[]/Path)	SHA-256 checksum	Path 는 IOException 가능
CpfIds	uuid, compactUuid, uuid32, normalizeSystemCode, temporaryId	일반 기술 ID	거래 transactionId 생성용이 아님; systemCode 3 자리 검증
CpfJson	write, read, map, list, toMap, convert	JSON↔DTO/Map/List	직렬화/역직렬화 실패는 IllegalArgumentException
CpfLists	emptyOrNull, distinct, partition, firstOrNull, lastOrNull, concat, page	immutable List·chunk·간단 page	chunkSize≤0, page<0/size<1 은 예외
CpfMaps	emptyOrNull, string, indexBy, getOrDefault, mutableCopy, stringMap	순서 보존 immutable Map·index	중복 key 이면 IllegalArgumentException 발생

Class	주요 Method	사용 목적	오류·경계
CpfNumbers	toInt, toLong, clamp	숫자 파싱·범위 제한	파싱 실패는 Optional empty; min>max 는 예외
CpfPages	request, page, offsetPage, slice	표준 Page/Slice 생성 편의	DB에서는 offsetPage 메모리 절단 대신 LIMIT/OFFSET 사용
CpfValidation	require, requireText, maxLength, notEmpty	Controller/Service 방어 검증	조건 위반은 IllegalArgumentException

2.1 대표 사용 예

```
import com.cpf.core.api.util.CpfStrings;
import com.cpf.core.api.util.CpfDates;
import com.cpf.core.api.util.CpfDecimals;
```

```
String customerNo = CpfStrings.requireText(input.customerNo(), "customerNo");
LocalDate baseDate = CpfDates.parseBasic(input.baseDate());
BigDecimal amount = CpfDecimals.scale(CpfDecimals.parse(input.amount()), 2, RoundingMode.HALF_UP);
```

`CpfIds`의 UUID 계열은 일반 기술 식별자입니다. End-to-End 거래 `transactionId`는 Transaction Context/Generator 계약을 사용하여 `CpfIds.uuid()`로 대체하지 않습니다.

2.2 Utility 선택 기준

상황	권장	피해야 할 구현
공백 입력 필수 검증	CpfStrings.requireText	각 Controller마다 trim/null if 반복
금액 scale	CpfDecimals.scale	setScale 을 화면/Repository 마다 제각각 적용
파일명/Root 하위경로	CpfFiles.safeFileName / resolveChild	사용자 입력을 Path.resolve 에 바로 연결
JSON DTO 변환	CpfJson	업무 코드마다 ObjectMapper 새로 생성
중복 key Map 생성	CpfMaps.indexBy	뒤 값으로 덮어쓰며 데이터 손실 숨김
대량 List 분할	CpfLists.partition	chunkSize 0/음수 예외 처리를 호출부마다 반복

3. Paging Reference - Offset, Slice, Sort, Cursor

API	계약	기본/제한	개발 포인트
CpfPageRequest	record(page,size), of(Integer,Integer), offset()	0-base, default 20, size 1..200, Math.multiplyExact	Controller에서 nullable 입력 정규화 후 Repository LIMIT/OFFSET 에 사용
CpfPage<T>	content,totalElements,page,size,totalPages,hasNext,items,hasPrevious,of	total>=0, page>=0, size>=1	COUNT 가 필요한 일반 목록 응답
CpfSlice<T>	items,page,size,hasNext,fromLookAhead	Repository 는 size+1 조회	COUNT 가 비싼 목록
CpfSort	field,direction, asc/desc	direction null→ASC	field 는 반드시 Repository allow-list 에 매핑
CpfSortDirection	ASC/DESC, from(String)	null/blank→ASC, 그 외 잘못된 값 예외	사용자 문자열을 직접 SQL 에 붙이지 않음
CpfCursor	opaque base64 value	blank 금지	외부 API 보안용은 Codec 사용
CpfCursorPage<T>	items,nextCursor,hasNext	hasNext=true 이면 nextCursor 필수	Keyset 응답
CpfCursorCodec	encode/decode	서명 검증 계약	외부 Cursor 는 단순 Base64 직접 생성 금지
CpfHmacCursorCodec	HMAC-SHA256 v1	secret 최소 32 byte	Secret Manager/환경 주입, rotation 은 Adapter

3.1 Offset Page Controller → Repository → 응답 조립

```
CpfPageRequest pageRequest = CpfPageRequest.of(page, size);
CpfSort sort = new CpfSort(sortField, CpfSortDirection.from(direction));

String orderBy = SORT_ALLOW_LIST.get(sort.field()); // allow-list 필수
List<CustomerRow> rows = repository.findCustomers(filter, pageRequest.offset(), pageRequest.size(), orderBy,
sort.direction());
long total = repository.countCustomers(filter);
return CpfPage.of(rows, pageRequest, total);
```

- `sort.field()`는 SQL 식별자이므로 요청 문자열을 `ORDER BY`에 직접 이어 붙이지 않습니다.
- `offset()`은 overflow를 조용히 감추지 않고 `Math.multiplyExact` 예외를 발생시킵니다.
- 대량 Export는 화면 Paging API 반복 호출로 대신하지 말고 Export Job/Batch 경계와 권한·감사를 별도로 설계합니다.
- Cursor payload에는 tenant/data scope·정렬 key가 필요한 경우 함께 서명해야 하며 다른 Scope의 Cursor 재사용을 허용하지 않습니다.

3.2 Cursor 위변조 처리

```
byte[] secret = secretProvider.get("customer-cursor-hmac");
CpfCursorCodec codec = new CpfHmacCursorCodec(secret);
String token = codec.encode(customerNo + "|" + createdAt);

// 다음 요청
String payload = codec.decode(token); // 서명/형식 오류는 IllegalArgumentException
```

4. 서비스 간 HTTP Public API - CpfHttpClient

패키지 `com.cpf.core.api.http.CpfHttpClient`. Generated Domain 이 서비스 간 HTTP 호출을 할 때 사용하는 공개 계약이며, ServiceCallEngine/Registry/retry/circuit/header propagation 구현을 업무 코드가 직접 생성하지 않습니다.

Method 형태	용도	주의
get(serviceld, uriFunction, Class<T>)	일반 GET	serviceld 는 Registry 계약 기준
get(standardExecutionId, serviceld, uriFunction, Class<T>)	표준 실행 ID 를 함께 사용하는 GET	Execution 정책과 Trace 연결
get(..., ParameterizedTypeReference<T>)	List/Generic 응답	Generic 타입 보존
post(serviceld,path,requestBody,Class<T>)	POST	비멩등 Retry 는 Idempotency/UNKNOWN 규칙 적용
post(...,ParameterizedTypeReference<T>)	Generic POST 응답	Error/Timeout 계약을 호출부에서 임의 치환 금지

```
CustomerResponse response = cpfHttpClient.get(
    "customer-service",
    uri -> uri.path("/api/customers/{id}").build(customerId),
    CustomerResponse.class);
```

완료 계약 HTTP 공개 API 는 timeout budget, connection pool, retry eligibility, masking, trace/metric, Provider 오류 매핑까지 실제 Consumer 경로에서 동일하게 적용되어야 합니다. 이 중 하나라도 누락되면 외부연계 완료로 판정하지 않습니다.

5. 신규 업무 Domain 개발 순서

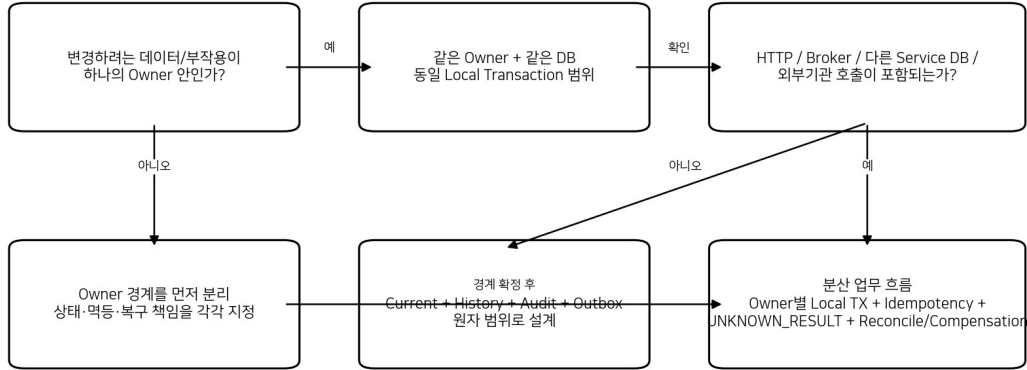
1. 업무 Owner 와 Aggregate/상태 전이를 먼저 정의합니다. “어느 서비스가 최종 상태를 소유하는가”가 Transaction/Recovery 기준입니다.
2. Query 와 Command 를 분리하고 입력 DTO 의 필수값·Default·Range·정렬 allow-list 를 정의합니다.
3. Application Service 가 하나의 Local Transaction 에서 보장할 Current/History/Audit/Outbox 범위를 결정합니다.
4. Persistence Port 와 Provider Adapter 를 연결하고 DB Migration/Index/Unique/Expected Version 을 정의합니다.

- Remote/Message/File/외부기관 호출은 Local Transaction 밖의 분산 경계로 설계합니다.
- Idempotency Key·Request Hash·Operation/Attempt 와 응답 유실 시 조회할 상태 원장을 정의합니다.
- Error Taxonomy 와 409/422/429/5xx 경계를 정하고 OpenAPI/Generated Client 를 갱신합니다.
- 정상·중복·동시성·Timeout·응답 유실·부분 실패·재처리·대사 Test 를 작성합니다.
- ADM 에서 조회할 transactionId/상태/Operation 과 운영 인계 절차를 연결합니다.

6. Transaction 판단 흐름 - 어디까지 묶고 언제 분리하는가

온라인 Transaction 경계 판단

외부 경계를 하나의 DB Transaction으로 가장하지 않습니다. Commit 이후 응답 유실은 rollback이 아니라 결과 불명일 수 있습니다.



상황	Local TX 에 포함?	실패 시	응답 유실/부분 실패 시
같은 Owner DB 의 Current + History + Audit + Outbox	가능	동일 DB rollback	DB commit 결과로 확정
다른 Service HTTP	불가	호출 대상의 commit 은 별도	대상 Operation 조회→Reconcile; 필요 시 Compensation
Broker publish 직접 전송	Local DB 와 같은 TX 로 가정 금지	Outbox commit 후 별도 publish	Outbox 상태/attempt/DLQ 대사
다른 Service DB	불가	각 Owner Local rollback 만	업무 상태기계로 정상화
외부기관 TCP/SFTP/API	불가	내부 rollback 과 외부 결과 분리	Tracking ID/전문번호/파일 checksum 으로 조회
Batch Chunk	Chunk 범위 내부만	현재 Chunk rollback	이전 Commit Chunk 는 유지, Restart/Reprocess

7. 상황별 Online 오류·Rollback Decision Matrix

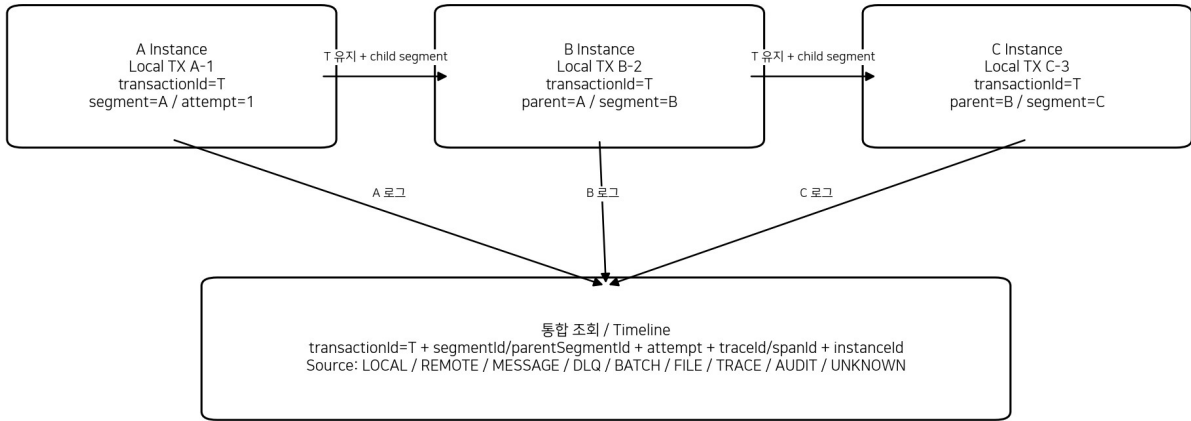
상황	CPF 판정	Rollback 범위	Retry 가능 조건	정상화
Validation 실패, DB 변경 전	명확한 실패	현재 Local TX 없음/rollback	입력 수정 후 가능	오류 코드 반환
DB optimistic conflict	Conflict	현재 Local TX rollback	최신 Version 재조회 후 새 의도로만	409 + expected/current version
Local DB deadlock	재시도 가능한 기술 실패 후보	현재 Local TX rollback	업무 역등성 + 제한된 retry budget	재실행 후 상태 확인
Remote connect 실패, 요청 미전송 확인	명확한 전송 실패	호출자 Local TX 정책대로	정책/역등성 충족 시 retry	Remote health 확인
Remote timeout, 대상 결과 불명	UNKNOWN_RESULT	대상 commit 여부 모름	blind retry 금지	Operation/Idempotency 조회→Reconcile

상황	CPF 판정	Rollback 범위	Retry 가능 조건	정상화
대상 Commit 성공 후 응답 유실	UNKNOWN_RESULT	대상 rollback 아님	같은 idempotency identity 로 조회 우선	CONFIRMED_SUCCESS 후 후속 처리
후속 Owner 성공 뒤 선행 Owner commit 실패	부분 성공	성공 Owner 는 유지	재호출보다 업무 Compensation 판단	Compensation + Audit
Outbox commit 후 broker 장애	전송 지연	업무 Local TX 는 이미 commit	Outbox publisher retry	Outbox/attempt/DLQ 대사

8. A → B → C 다중 인스턴스 Transaction / Log

A → B → C 다중 인스턴스 Transaction / Log 계보

C가 Commit 후 응답이 유실되면 C를 rollback했다고 추정하지 않습니다. Operation/Attempt를 조회해 Reconcile하고 필요하면 Compensation을 수행합니다.



A/B/C 가 서로 다른 Instance·Owner 라면 `A-1 Local TX → B-2 Local TX → C-3 Local TX`입니다. 세 구간을 하나의 물리 DB Transaction 으로 설명하지 않습니다. 전체 업무는 root `transactionId=T`를 유지하고 실행 구간은 `segmentId / parentSegmentId / attempt / traceId / spanId / instanceId`로 계보를 만듭니다.

시나리오	무엇이 rollback 되는가	로그/상태 해석	개발자 조치
C 가 Commit 전 실패	C Local TX	C segment 실패, B/A 가 오류 수신	B/A 정책에 따라 Local rollback 또는 상태 전이
C Commit 성공 후 응답 유실	C 는 이미 성공 가능	B 에는 UNKNOWN_RESULT, C source 는 성공 기록 가능	같은 transactionId + Operation/Attempt 로 C 결과 조회
B 가 C 성공 확인 후 B Commit 실패	B 만 rollback, C 는 유지	C 성공 segment + B 실패 segment 공존	업무상 필요하면 C Compensation
A 가 B 성공 후 실패	A 만 rollback, B/C 유지 가능	root T 아래 부분 성공 계보	B/C 상태 조회 후 Compensation/forward recovery
일부 Source 조회 실패	거래 전체 성공으로 숨기지 않음	APPLICABLE source 가 FAILED/MISSING/STALE	운영자가 source freshness 확인

운영자가 transactionId 로 추적 → [05 플랫폼운영매뉴얼.pdf](#) / §4 Transaction Timeline / Source Freshness

완료 계약 first-hop transactionId 는 인증된 최초 기동 System 과 durable replay identity 가 확인된 경우에만 보존하고, 이후 A→B→C hop 에서는 transactionId 를 유지하면서 segment/parent/attempt/trace 를 추가합니다. 신뢰 근거가 없으면 inbound transactionId 를 수용하지 않습니다.

8.1 Inbound transactionId trust boundary

- TransactionContextFilter 는 HIGHEST_PRECEDENCE 에서 동작하며 CpfInboundTransactionIdPolicy 로 inbound ID 를 판정합니다.
- ID 가 없으면 Local Start 로 새 transactionId 를 생성합니다.
- ID 가 있으면 형식 검증뿐 아니라 trusted transport, authenticated system provenance 를 요구합니다.

- 정식 Channel Start 는 requestId/idempotency/externalRequestId 중 하나를 replay identity 로 요구하고 durable idempotency claim 을 사용합니다.
- 동일 시작 요청의 retry 는 같은 transactionId 를 보존할 수 있지만, 다른 요청이 기존 ID 를 replay 하여 계보에 합류하는 것은 거부합니다.
- 응답에는 transactionId / traceId / spanId 가 설정됩니다.

9. Idempotency / UNKNOWN_RESULT / Reconcile / Compensation

개념	언제 사용	필수 식별자	금지
Idempotency	같은 업무 요청의 중복 전달	scope + idempotencyKey + requestHash	key 만 같고 payload 가 다른 요청을 같은 요청으로 처리
Operation/Attempt	원격/비동기 실행 회차 추적	operationId + attempt + owner/target	응답 유실 후 새 ID 로 blind retry
UNKNOWN_RESULT	요청이 도달/commit 했는지 모름	transactionId + operation/idempotency identity	실패로 단순 치환
Reconcile	Owner 의 실제 상태를 조회해 결과 확정	Owner query + immutable identity	문자열 substring 으로 다른 실행 선택
Compensation	이미 commit 된 다른 Owner 의 업무 효과 취소/정정	원거래 reference + reason + approval/audit	DB rollback 인 것처럼 설명

1. Timeout/response loss 를 감지하면 현재 Local TX 결과와 Remote 결과를 분리합니다.
2. 같은 idempotency/operation identity 로 Owner 상태를 조회합니다.
3. Confirmed Success 면 후속 Local 상태를 완료시키거나 누락된 projection/outbox 를 복구합니다.
4. Confirmed Failure 면 호출자 정책에 따라 재실행 또는 실패 확정합니다.
5. 계속 Unknown 이면 수동 Review/승인 경계로 올립니다.
6. 부분 성공을 업무적으로 되돌려야 하면 Compensation Command 를 별도 거래로 실행하고 Audit 합니다.

10. Messaging - Outbox / Inbox / DLQ

- 업무 DB 변경과 Event 발행 의도를 같은 Local Transaction 에 보장하려면 Outbox 를 사용합니다.
- Broker publish 성공 자체를 업무 DB commit 과 같은 Transaction 으로 설명하지 않습니다.
- Consumer 는 messageId/businessKey/requestHash 등으로 중복 수신을 방어하는 Inbox/Idempotency 경계를 둡니다.
- 처리 실패는 retry budget, DLQ, poison message, Reprocess 와 업무 대사 기준을 구분합니다.
- transactionId 는 Message header 에도 승계하고 새 consumer segment 를 만들어 parent 관계를 유지합니다.

11. File / Attachment / 외부기관 연계

연계	Transaction 경계	식별/대사	실패 복구
Attachment/File upload	DB metadata 와 실제 storage write 분리 가능	fileId, checksum, size, owner transactionId	orphan/partial upload cleanup, 재검증
SFTP batch/file	외부 server 와 Local TX 분리	fileName, checksum, remote path, transfer attempt	temp→rename, 재전송, 수신 대사
HTTP 기관	Remote Owner/기관 별도 commit	requestId, providerTrackingId, transactionId	timeout→status query/reconcile
TCP/전문	전문 송수신과 DB commit 분리	전문번호, sequence, response code	응답 유실→조회전문/대사
대량 Export	온라인 Page API 와 분리	exportJobId, scope, audit	Job 재시작/재생성, 만료/권한 검증

12. Security / Audit / Secret / Data Scope

- Authorization 은 서버 Owner 경계에서 최종 판정하고 UI 버튼 숨김만으로 권한을 대체하지 않습니다.
- 민감정보는 API·로그·DB summary·Evidence 경계에서 Masking/Redaction 을 적용하고 Secret/Token/PII 원문을 기록하지 않습니다.
- 위험 조치는 Permission + Data Scope + Reason + Approval + Expected Version + Idempotency + Audit 을 함께 판단합니다.
- Secret 은 Source 나 문서 예제에 평문으로 넣지 않고 Secret Provider/환경 주입을 사용합니다.

- Signed Cursor HMAC key 도 Secret 관리 대상이며 최소 32 byte 계약을 따릅니다.

13. 개발자가 Log 를 어떻게 남겨야 하는가

개발자 책임	Framework 책임	하지 말 것
Transaction Context 를 끊지 않고 transactionId/trace/segment 의미를 유지	Header propagation, Context lifecycle, 내부 writer/trace bridge	CpfFileLogWriter/RecoverySpool 같은 Internal 구현 직접 new
업무 의미가 있는 상태·오류·식별자를 구조화하여 남김	Masking, persistence/file sink, timeline aggregation	Secret/PII/raw payload 무제한 기록
Operation/Attempt/Idempotency identity 를 Remote/Async 호출과 연결	통합 Timeline source 수집	응답 유실을 단순 ERROR 한 줄로 끝냄
Compensation/Reconcile 결과와 reason/audit reference 기록	운영 조희/Source freshness	로그 존재만으로 거래 성공 판정

FileLog/Spool/운영 장애 정본 → [05 플랫폼운영매뉴얼.pdf](#) / §4~§6

14. 환경·Build·Generator·DB Migration·OpenAPI·Test·인계

단계	개발자가 끝내야 할 일	정상 판정	실패 시
환경/Build	Java/Gradle/Profile/Starter 조합 확인	대상 Module clean compile/test	toolchain 불일치 시 Build Gate 실패로 처리하고 지원 Toolchain 으로 교정
Generator	Profile/Capability/Provider 선택, 생성 Manifest/Starter Lock 확인	생성물 Source/Config/Migration/Test 일치	수기 의존 추가로 Generator 정합성 깨지지 않게 재생성/비교
DB Migration	Vendor SQL/Index/Unique/rollback 전략	Fresh install + migration + drift 점검	rollback/restore 또는 후속 corrective migration
OpenAPI	Controller/Error DTO/operationId 와 checked-in spec 동기화	Generated Client 와 Runtime route parity	Generated client 수기 수정 금지
Test	정상/Validation/Conflict/Idempotency/Timeout/Unknown/Compensation	실행된 Test evidence	지원 환경별 Test Matrix 를 실행하고 Evidence 를 인수
ADM 확인	transactionId/상태/operation/audit 가 조희되는지	운영자가 원인과 정상화 판정 가능	source freshness/권한/route 를 분리 점검
배포 인계	Artifact/Config/Secret/Migration/Runbook/rollback 전달	운영팀이 문서만으로 설치·복구 가능	05 매뉴얼과 불일치하면 인계 중단

14.1 실전 기능 카드 - Utility 를 업무 코드에 적용하는 기준

문자열·Validation - 입력 정규화와 오류 경계

필수 입력은 Controller 에서 한 번 검증하고, Domain 안에서 같은 null/blank 방어 코드를 반복하지 않습니다. 다만 업무 규칙 검증과 기술 입력 검증을 한 메서드에 섞지 않습니다.

```
import com.cpf.core.api.util.CpfStrings;
import com.cpf.core.api.util.CpfValidation;
```

```
String accountNo = CpfStrings.requireText(request.accountNo(), "accountNo");
CpfValidation.maxLength(accountNo, "accountNo", 30);
CpfValidation.require(request.amount() != null, "amount 는 필수입니다.");
```

상황	사용	결과	주의
null/blank 필수값	CpfStrings.requireText	trim 후 값, 없으면 IllegalArgumentException	업무 Error DTO 로 변환하는 책임은 Web/Error 계층
선택값 blank→null	CpfStrings.trimToNull	null 또는 trim 값	빈 문자열을 유효 코드로 보존해야 하는 업무에는 사용 금지

상황	사용	결과	주의
길이 제한	CpfValidation.maxLength	초과 시 예외 발생	DB column 길이와 OpenAPI maxLength 도 일치
코드 대문자화	normalizeCodeOrNull	null 보존 + upper-case	Locale.ROOT 계약 유지

날짜·시간 - 업무 기준일과 시스템 시각을 분리

```

LocalDate baseDate = CpfDates.parseBasic("20260808");
String externalDate = CpfDates.format(baseDate);
Instant requestedAt = CpfTimes.now();
LocalDateTime local = CpfTimes.toLocalDateTime(requestedAt, ZoneId.of("Asia/Seoul"));

```

- `CpfTimes.today()` / `nowDateTime()` 은 시스템 현재 시각을 사용합니다. 거래의 업무 기준일·회계일·Batch 기준일을 “현재 시각”으로 대체하지 않습니다.
- ZoneId 가 필요한 변환에는 명시적으로 전달합니다. 서버 기본 timezone 에 의존하면 다중 Region/DR 환경에서 결과가 달라질 수 있습니다.
- `parseOrNull()` 은 잘못된 값도 null 로 돌려주므로 “입력 오류를 반드시 거부해야 하는 API”에서는 `parse()` 를 사용합니다.

금액·숫자 - 금융 계산에서 scale 과 rounding 을 숨기지 않는다

```

BigDecimal raw = CpfDecimals.parse(request.amount());
BigDecimal settled = CpfDecimals.scale(raw, 2, RoundingMode.HALF_UP);
int pageSize = CpfNumbers.clamp(inputSize, 1, 200);

```

- 통화/상품별 scale 과 rounding 이 업무 규칙이면 호출부에서 의미 있는 정책 이름과 함께 명시합니다.
- `zeroOrNull()` 은 null 과 실제 0 을 동일하게 취급해도 되는 집계에서만 사용합니다. “미입력”과 “0 원”이 다른 업무에는 사용하지 않습니다.
- `CpfNumbers.toInt/toLong` 은 파싱 실패를 Optional empty 로 반환하므로 필수 입력이면 별도 Validation 을 결합합니다.

파일·Hash - 경로 traversal 과 무결성 확인

```

String safeName = CpfFiles.safeFileName(upload.originalName());
Path target = CpfFiles.resolveChild(uploadRoot, safeName);
String checksum = CpfHashes.sha256(target);

```

검사	정상	오류/복구
safeFileName	단일 파일명만 허용	/ 또는 \ 포함, ../ .. 는 거부; 원본 이름을 그대로 경로로 사용하지 않음
resolveChild	정규화 후 root 하위	root 밖으로 벗어나면 예외; symlink/권한은 storage 계층에서 추가 검증
sha256(Path)	파일 checksum 산출	IOException 은 업로드 성공으로 숨기지 않고 artifact 를 실패 상태로 확정된 뒤 Retry/정리 절차 수행

JSON·Collection - 편의 API 가 업무 의미를 숨기지 않게 사용

```

Map<String, Object> payload = CpfJson.toMap(command);
List<List<CustomerId>> chunks = CpfLists.partition(customerIds, 500);
Map<String, Customer> byId = CpfMaps.indexBy(customers, Customer::id);

```

- `CpfJson` 은 DTO 변환 편의 API 이지 외부 입력 Schema 검증을 대체하지 않습니다.
- `CpfMaps.indexBy` 는 중복 key 를 조용히 덮어쓰지 않고 실패합니다. 중복이 정상인 데이터라면 Map 이 아니라 Multi-value 모델을 사용합니다.
- `CpfLists.partition` 은 메모리 List 분할입니다. DB 대량 조회/Batch 를 이 메서드 하나로 대체하지 않습니다.

14.2 Paging 실전 선택 - Page / Slice / Cursor / Export

요구	선택	Repository 조회	응답	왜
총 건수와 페이지 번호 필요	CpfPage	COUNT + LIMIT/OFFSET	totalElements/totalPages	관리 목록·검색 UI

요구	선택	Repository 조회	응답	왜
총 건수 불필요, 다음 페이지 여부만	CpfSlice	size+1 LIMIT/OFFSET	hasNext	COUNT 비용 절감
데이터 변경이 잦고 깊은 페이지	CpfCursorPage + Signed Cursor	keyset WHERE + ORDER	nextCursor	offset drift/대량 페이지 회피
수십만~수백만 건 Export	Batch/Export Job	별도 Snapshot/Job	artifact/jobId	온라인 Paging 반복 호출로 장시간 세션 점유 금지

Page 오류 계약과 운영 확인

입력/상황	Framework 반응	API 계층 처리	운영 확인
page < 0	IllegalArgumentException	4xx validation error 로 표준화	오류율/잘못된 client 확인
size < 1 또는 > 200	IllegalArgumentException	4xx + 허용 범위 안내	client version 확인
page*size overflow	ArithmeticException	잘못된 paging 요청으로 거부	비정상/공격 입력 여부
sort field 미허용	업무 allow-list 에서 거부	4xx	허용 필드와 OpenAPI 문서 일치
cursor signature 불일치	IllegalArgumentException	4xx, 내부 payload 미노출	반복 실패는 tamper/잘못된 key rotation 조사
cursor scope 불일치	업무 계층에서 거부	403/409 등 계약에 맞게	tenant/data-scope audit

Signed Cursor payload 설계 예

```
// 예시 payload 의미: tenant + sort key + PK + version
String payload = tenantId + "|" + createdAt + "|" + customerId + "|v1";
String cursor = codec.encode(payload);
```

```
// 다음 요청에서는 서명 검증 후 현재 사용자의 tenant/data scope 와 다시 대조
String decoded = codec.decode(cursor);
```

Cursor 토큰이 유효하게 서명됐다는 사실은 “현재 사용자가 그 데이터 Scope 를 볼 권한이 있다”는 뜻이 아닙니다. 서명 검증과 Authorization/Data Scope 검증은 별도입니다.

14.3 Online Transaction Worked Cases - 개발 중 바로 판단하는 4 개 사례

Case A - 한 Owner DB 에서 Current + History + Audit + Outbox

1. Application Service 시작 시 하나의 Local Transaction 을 엽니다.
2. Current row 를 Expected Version 으로 갱신하고 History/Audit/Outbox 를 같은 DB Transaction 안에 기록합니다.
3. Outbox record 까지 Commit 되면 “업무 변경 의도”는 원자적으로 확정됩니다.
4. Broker publish 는 Commit 이후 별도 Publisher 가 수행합니다. Broker 장애가 나도 이미 Commit 된 업무 DB 를 rollback 됐다고 쓰지 않습니다.
5. 운영자는 업무 row + Outbox 상태를 대사하고 Publisher retry/DLQ 를 확인합니다.

Case B - A 가 B 를 HTTP 로 호출하고 B 가 Commit 후 응답 유실

1. A 는 `transactionId=T`, idempotency/operation identity 를 B 로 전달합니다.
2. B 는 자체 Local Transaction 을 Commit 하고 B segment/operation 상태를 남깁니다.
3. 응답이 A 에 도달하지 않으면 A 는 B 실패로 확정하지 않고 `UNKNOWN\_RESULT`로 전이합니다.
4. A 는 동일 identity 로 B 상태를 조회합니다. Confirmed Success 면 A 의 후속 상태를 진행하고, Confirmed Failure 면 retry 가능 조건을 평가합니다.
5. 계속 Unknown 이면 수동 Review/승인 또는 정해진 Reconcile job 으로 넘깁니다.

Case C - A→B→C 에서 C 성공 후 B Commit 실패

1. C 성공은 C Owner 의 Local Commit 이므로 B rollback 으로 되돌아가지 않습니다.
2. B 는 자신의 Local Transaction 을 rollback 하고 root T 아래 B failure + C success 를 함께 남깁니다.
3. 업무상 C 효과를 취소해야 하면 C 의 Compensation Command 를 별도 거래로 실행합니다.

4. Compensation 도 실패/응답 유실 가능하므로 새 operation/attempt 와 원거래 reference 를 남깁니다.
5. 운영 정상화는 "B rollback 완료"가 아니라 B/C 의 최종 업무 상태와 Audit 까지 확인해야 끝납니다.

Case D - 동일 요청이 재전달되었으나 payload 가 달라짐

1. scope + idempotencyKey 가 같더라도 requestHash 가 다르면 같은 요청으로 재사용하지 않습니다.
2. 기존 결과를 그대로 반환하거나 새 요청을 덮어쓰면 데이터 오염이 되므로 conflict/replay violation 으로 거부합니다.
3. 운영 로그에는 key 자체뿐 아니라 request hash mismatch 와 caller/transaction identity 를 남깁니다.
4. 사용자/외부기관이 진짜 새 의도로 다시 요청하려면 새 idempotency identity 를 발급합니다.

14.4 개발 로그 필드 Checklist - A→B→C 를 실제로 찾을 수 있게

필드	언제 반드시 연결	목적	주의
transactionId	거래 최초 시작부터 모든 hop	전체 업무 root 검색	일반 UUID utility 로 임의 재생성 금지
segmentId / parentSegmentId	Owner/Remote/Async 실행 구간	A→B→C 계보	parent 누락 시 root 만 같고 순서 복원 불가
attempt	retry/takeover/reprocess	실행 회차 분리	같은 side effect 중복 여부와 함께 해석
traceld / spanId	동기/비동기 기술 호출	latency/호출 경로	sampling 누락과 거래 실패를 구분
operationId	비동기/원격 명령	UNKNOWN 상태 조회	새 retry 마다 의미 없는 새 operation 남발 금지
idempotencyKey / requestHash	중복 방지 Command	동일 요청 검증	민감 payload 자체를 log 에 기록하지 않음
instanceId / workerId	다중 인스턴스/Batch	실행자 ownership	lease/fence 와 함께 해석
errorCode / state	모든 실패/상태 전이	자동 분류/운영 판단	stack trace 만 남기지 않음
reason / approvalRef / auditRef	위험조치·Compensation	감사 추적	PII/Secret 원문 금지

Source Trace / 구현 근거

구분	실제 경로·계약	현재 판정
Utility	cpf-core/src/main/java/com/cpf/core/api/util/**	Public Utility 는 실제 Consumer 와 경계 Test 까지 계약으로 유지
Paging	cpf-core/src/main/java/com/cpf/core/api/page/**	PageRequest/Page/Sort/Cursor Source 확인
HTTP	cpf-core/src/main/java/com/cpf/core/api/http/CpfHttpClient.java	Public 계약과 Provider Runtime 의 timeout/retry/metric/error mapping 을 함께 유지
transactionId policy	cpf-core/.../CpfInboundTransactionIdPolicy.java + TransactionContextFilter.java	first-hop trust producer 와 multi-hop propagation 이 동일 transactionId 계약을 준수
A→B→C executable sample	온라인→원격→결과대사 통합 시나리오	정상·응답 유실·중복·보상 시나리오를 통합 Acceptance Test 로 실행
Timeline freshness	CpfTransactionTimelineQueryFacade	Source query 예외는 FAILED, 미적용 Source 만 NOT_APPLICABLE 로 분리

15. Definition of Done - 온라인 기능

- 기능의 Owner, Public API/SPI/Internal 경계를 문서와 Source 에서 동일하게 확인했습니다.
- 입력·기본값·Range·Sort allow-list·Data Scope·Permission 을 정의했습니다.
- Local Transaction 범위와 Remote/Message/File 경계를 그림 또는 표로 설명할 수 있습니다.
- 정상/Conflict/Timeout/응답 유실/부분 성공/UNKNOWN_RESULT 에 대해 Rollback·Retry·Reconcile·Compensation 판단이 있습니다.

- transactionId/segment/attempt/trace 가 A→B→C 계보를 유지하고 운영자가 05/ADM 에서 확인할 수 있습니다.
- 실행한 Test 와 실행하지 못한 Runtime 을 구분 기록했습니다.
- 다른 매뉴얼에 동일 설명을 복사하지 않고 필요한 경우 정보 절 링크만 추가했습니다.

18. Starter / Capability Profile 정식 구조

구분	현재 정보	개발자가 판단할 것
공개 Profile	minimal-domain, web-api, secure-api, browser-bff, event-service, batch-service	업무 유형으로 시작하고 resolved starter lock 을 확인
Capability Group	data, messaging, integration, file, notification, security, platform-operations	Profile 만으로 부족한 기술 capability 를 추가 선택
Data	JDBC/MyBatis, Caffeine/Valkey provider	DB 접근 방식과 Cache Provider 를 선택
Messaging	Kafka/RabbitMQ/JMS/IBM MQ + reliability JDBC	Broker/legacy MQ 요구와 Outbox/Inbox 의미를 구분
Integration	HTTP Client, resilience, TCP, fixed-length, ISO8583, webhook	상대기관 Protocol/timeout/idempotency 에 따라 선택
File	Attachment, Archive, Tabular/POI, SFTP	파일 수명주기와 외부 전달 요구에 따라 선택
Security	Resource Server, JDBC Session, Service Identity, Secret	사용자/서비스 인증과 Secret Provider 를 분리
Platform Ops	Observability/OTLP, Runtime Control Client, Channel Registry, Feature Flag	운영/관측 기능은 업무 코드와 분리

정본 Source: cpf-tools/generator/contracts/cpf-starter-catalog.json, settings.gradle. 과거 Starter 이름이나 과거 Profile 수를 문서 정보로 사용하지 않습니다.

19. 외부연계 선택표와 Public 진입점

요구	선택	Public 진입/계약	오류·복구 핵심
내부/외부 REST 호출	HTTP Client capability	com.cpf.core.api.http.CpfHttpClient	timeout 후 부작용 여부가 불명확하면 UNKNOWN_RESULT/Reconcile
장기 TCP 세션/전문	TCP capability	integration/tcp 의 공개 계약과 Provider	connection reset/half-open/재전송 시 역등기와 attempt 필요
고정길이 전문	fixed-length capability	integration/fixedlength	layout/version/encoding 오류를 요청 오류와 전송 오류로 구분
ISO8583	ISO8583 capability	integration/iso8583	MTI/field validation, reversal/timeout 정책을 업무 원장과 연결
파일 송수신	SFTP + File capability	file/sftp + File/Attachment API	업로드 완료 전 접근 금지, checksum/건수 대사
비동기 Broker	Messaging capability	Kafka/JMS/Rabbit/IBM MQ Provider + 공동 메시지 계약	Outbox/Inbox/DLQ, duplicate/replay/reconcile
Webhook	Webhook capability	integration/webhook	delivery attempt, DLQ, replay, UNKNOWN 구분
알림	Notification capability	notification dispatch/provider	업무 성공과 알림 성공을 분리

20. CpfHttpClient 실제 사용 예

```
import com.cpf.core.api.http.CpfHttpClient;
```

```
CustomerResponse response = cpfHttpClient.get(
    "customer-service",
    uri -> uri.path("/api/customers/{id}").build(customerId),
    CustomerResponse.class);
```

- servidId 는 Service Registry/승인된 대상과 연결합니다. URL 을 업무 코드에 하드코딩하지 않습니다.
- GET/POST 및 ParameterizedTypeReference overload 가 공개 계약에 존재합니다.
- HTTP 호출을 현재 DB Local Transaction 에 포함된 것처럼 설명하지 않습니다. 상대 Owner 의 Commit 과 응답 유실은 별도 결과입니다.

- 실제 retry/circuit/header propagation 은 내부 ServiceCallEngine/Registry 계층이 적용하며 업무 코드는 Public 계약을 사용합니다.

21. 개발 완료 전 질문 Checklist

질문	예이면	아니오/불명확이면
Public API/SPI 인가?	문서화된 계약만 import	Internal package import 제거
Transaction 경계가 한 Owner 인가?	Local TX 적용	Owner 별 Local TX + 복구 설계
중복 요청이 가능한가?	Idempotency key/hash 정의	재전송 금지 또는 명시적 결과 조회
응답 유실 후 결과를 아는가?	상태 확정	UNKNOWN_RESULT + Operation lookup
외부 부작용이 있나?	attempt/Provider ID 기록	단순 retry 여부 재검토
ADM 에서 확인해야 하나?	03 연동 계약과 Operation ID 연결	개발 로그/Metric 만으로 충분한지 결정

22. Public Utility Method Reference

Class	실제 Method	사용	경계
CpfStrings	hasText, trimOrNull, requireText, defaultIfBlank, normalizeCodeOrNull	문자열 정규화/필수 검증	blank 와 null 의미가 다른 업무는 주의
CpfDates	parse, parseBasic, parse(formatter), parseOrNull, format, formatBasic, betweenInclusive	LocalDate 입력/출력	parseOrNull 은 오류를 null 로 바꾸므로 필수 입력에 남용 금지
CpfTimes	now, now(zone), today, nowDateTime, nowDateTimeMillis, toInstant, toLocalDateTime	시각/Timezone 변환	업무 기준일을 시스템 now 로 대체 금지
CpfDecimals	parse, scale, zeroIfNull, isZero	금액/Decimal	scale/rounding 은 업무 규칙에 맞게 명시
CpfHashes	sha256(String/byte[]/Path)	무결성/Artifact Hash	Path 버전 IOException 처리 필요
CpfFiles	safeFileName, extension, resolveChild	Path traversal 방어	symlink/storage 권한은 별도
CpfIds	uuid, compactUuid, uuid32, normalizeSystemCode, temporaryId	기술 ID	transactionId 생성 대체 금지
CpfLists	emptyOrNull, distinct, partition, firstOrNull, lastOrNull, concat, page	Collection 편의	대량 DB/Batch 를 메모리 분할로 대체 금지
CpfMaps	emptyOrNull, string, indexBy, getOrDefault, mutableCopy, stringMap	Map 변환/Index	indexBy 중복 key 는 실패
CpfNumbers	toInt, toLong, clamp	숫자 parse/범위	Optional empty 처리 의미 확인
CpfPages	request, page, offsetPage, slice	Page/Slice 생성	offsetPage 는 메모리/교육용, DB 는 LIMIT/OFFSET
CpfValidation	require, requireText, maxLength, notEmpty	방어 검증	업무 규칙 Error taxonomy 와 연결
CpfJson	write, read, map, list, toMap, convert	JSON 변환	외부 Schema validation 대체 아님

23. Starter Leaf/Provider 선택 Reference

Starter	기능	언제 선택	핵심 경계
foundation/base	최소 Spring Boot 조립	모든 Profile 기반	업무/DB/Message 기능 강제 금지
data/persistence-jdbc	JDBC Persistence	SQL 중심 Repository	DB vendor/profile 필요
data/persistence-mybatis	MyBatis Persistence	Mapper 기반	BOM/MyBatis version 정렬
data/cache-caffeine	Local cache	single-instance/local cache	분산 일관성 필요 시 부적합
data/cache-valkey	분산 cache provider	shared cache	외부 Valkey 운영 필요
messaging/reliability-jdbc	Outbox/Inbox reliability	Broker 와 DB 사이 신뢰성	업무 DB transaction 경계 설계
messaging/kafka	Kafka provider	event/stream	topic/partition/idempotency
messaging/rabbitmq	RabbitMQ provider	고객 외부 연동/승인된 경우	제품 기본 broker 정책과 구분

Starter	기능	연제 선택	핵심 경계
messaging/jms	JMS	표준 JMS	provider 특성 확인
messaging/ibm-mq	IBM MQ	금융/legacy MQ	queue manager/channel 운영
integration/http-client	Service HTTP Client	REST/내부 Remote	Service Registry/timeout
integration/resilience	timeout/retry/CB/bulkhead	외부/Remote 호출	비역등 retry 제한
integration/tcp	TCP 연제	socket 전문	connection/reconnect
integration/fixedlength-core	고정길이 core codec	전문 layout	version/encoding
integration/fixedlength	고정길이 runtime	전문 송수신	message contract
integration/iso8583	ISO8583	금융 전문	reversal/timeout 업무 규칙
integration/webhook	Webhook	외부 callback	attempt/DLQ/replay
file/attachment	Attachment	업로드/다운로드	security scan/permission
file/archive	Archive	zip/archive	zip slip/bomb
file/tabular-poi	Excel/Tabular	업로드/Export	streaming/size
file/sftp	SFTP	기관 파일 연제	checksum/rename/atomicity
notification/dispatch	알림 dispatch	provider 추상화	업무 성공과 전달 성공 분리
notification/email	Email	SMTP/provider	retry/PII
notification/sms-spi	SMS SPI	고객 Provider 구현	credential/audit
security/resource-server	Resource Server	API token	issuer/audience
security/session-jdbc	JDBC Session	browser/session	TTL/revoke
security/service-identity	Service Identity	service-to-service	mTLS/HMAC trust
security/secret	Secret	secret provider	raw property/log 금지
platform-operations/observability	Micrometer/Observation	metric/trace/log correlation	SDK type Public API 노출 금지
platform-operations/otlp	OTLP exporter	외부 collector	endpoint/TLS/queue
platform-operations/runtime-control-client	Runtime Control	ADM 변경 적용	version/partial apply
platform-operations/channel-registry-jdbc	Channel Registry	채널/실행 정책	DB lifecycle
platform-operations/feature-flag-openfeature	Feature Flag	runtime flag	approval/kill switch

24. Generator 실무 순서

단계	입력/결과	확인
1 Profile 선택	minimal-domain/web-api/secure-api/browser-bff/event-service/batch-service	요구 Capability 와 맞는지
2 Domain Metadata	DomainName/SystemCode/package/artifact/schema/table prefix	고정 MBR/ACC 추정 금지
3 생성	cpf-tools/generator/create-domain.ps1	resolvedStarters/manifest 생성
4 DB	Generated Domain manifest 기반 DB init	platform pack 과 분리
5 Build/Test	생성 Domain build/test	Golden parity
6 Local/Remote	Facade/Remote 계약	Topology 변경 시 업무 source 재작성 금지
7 OpenAPI/JavaDoc	Public 계약 산출	operationId/문서
8 인계	Artifact/Config/Migration/Test/ADM	05 운영 인계

25. 외부연계 Fault Decision Matrix

연계	Fault	무조건 retry?	먼저 확인	복구
HTTP GET	connect fail	정책 내 가능	요청 미전송/timeout budget	retry/CB
HTTP Command	response timeout	아니오	operation/idempotency 결과	reconcile/compensation
Kafka/JMS/MQ	broker ack loss	아니오	Outbox/producer attempt/inbox	publisher retry/DLQ
TCP 전문	socket reset after send	아니오	전문번호/상대 tracking	조회/reversal/reconcile
SFTP	rename 전 실패	상황별	temp/final/checksum	resume/reupload
Webhook	2xx response loss	아니오	delivery id/attempt	status/replay
Notification	provider timeout	업무 성공과 분리	delivery ledger	retry/DLQ

26. Security/DB/OpenAPI/Test 인계 체크

축	개발자가 끝내야 할 것	정본
Security	권한/Data Scope/Masking/Secret/service identity	기술표준서
DB	migration/index/verify/rollback/DB3 의미	데이터베이스표준서
OpenAPI	operationId/schema/error/version	기술표준서
Test	unit/integration/concurrency/fault/recovery	기술표준서
ADM	operationId/route/상태/권한 연결	03
Operations	property/secret/health/log/rollback 인계	05

27. 질문 표현이 달라도 찾는 개발자 인덱스

질문에서 들리는 말	먼저 볼 곳	판단이 끝나야 하는 것
문자열·날짜·금액·Hash·파일명 변환	§22 Public Utility Method Reference	공개 Utility 인지 JDK/업무 규칙인지
목록 조회·총건수·다음 페이지	Paging Reference + §22 CpfPages	Page/Slice/Cursor 선택과 sort allow-list
서비스 호출·REST·timeout	§19~§20 외부연계/CpfHttpClient	GET/Command, timeout budget, UNKNOWN_RESULT
거래를 어디까지 묶나	Transaction 경계/Worked Case	Owner Local TX 와 외부 side effect 분리
중복 요청·재전송	Idempotency/UNKNOWN_RESULT	key/hash/result lookup 과 재처리 금지 조건
Kafka·JMS·IBM MQ·Rabbit	Starter/Capability + Messaging	Broker Provider 와 reliability/outbox 역할
TCP·고정길이·ISO8583	§19 외부연계 선택표	layout/version/reversal/attempt 기준
SFTP·첨부·Excel	File capability	temp/validated/final, checksum, size/security
Starter 가 뭐가 있나	§18 + §23 Starter Reference	Profile→Capability→Leaf/Provider 선택
새 Domain 을 만들고 싶다	§24 Generator 실무 순서	manifest/resolved starters/DB/Test/인계
ADM 예도 보여야 한다	03 ADM 개발자 매뉴얼	Owner Operation→Route/Generated Client 연결
배포 후 운영팀에 뭘 넘기나	§26 + 05 플랫폼 운영	Artifact/Config/Migration/Test/ADM/Runbook

28. 구현 시작 전 6 개 결정

- Owner 가 누구인지 정하고 다른 Owner DB 를 직접 갱신하지 않습니다.
- 조회인지 부작용 Command 인지 먼저 구분합니다.
- Local TX 안의 원자성 범위와 외부 side effect 를 분리합니다.
- 중복·동시성·timeout·response loss 때 결과 판정 키를 먼저 설계합니다.
- Public API/SPI 만 사용하고 Internal package 직접 의존을 제거합니다.
- 정상 결과뿐 아니라 Log/Metric/Trace/Audit 와 Recovery 판정 위치까지 개발 완료 조건에 넣습니다.

29. 온라인·외부연계 수용 판정 Gate

축	완료 조건	실패 시 돌아갈 절
Build/Generator	지원 Toolchain 에서 Build, Generator Dry Run/Apply, Manifest/Starter Lock 일치	§14 / §24
API/Validation	Public API/SPI 만 사용하고 입력·Page/Sort/DTO/Error 계약이 Test 로 고정	§1~§5
Transaction	Owner 별 Local TX 경계와 A→B→C 결과/보상/UNKNOWN 판단이 명확	§6~§9 / §14.3
외부연계	HTTP/TCP/전문/File/Message 별 timeout/idempotency/attempt/recovery 가 정의	§10~§13 / §19~§25
Security/Audit	Service/User identity, Permission/Data Scope, Secret/PII Masking, Audit 가 적용	§12 / §26
DB/OpenAPI/Test	Migration/rollback, OpenAPI/generated client, 정상·오류·Fault Test 가 일치	§14 / §26
운영 인계	ADM 확인 위치, Log/Metric/Trace, Runbook, rollback/reconcile 기준을 운영자에게 전달	§13 / §26

30. 온라인 개발 인수 Evidence Record

축	인수할 Evidence
Build/Generator	toolchain, build result, dry-run/apply result, manifest, starter lock
API	OpenAPI/JavaDoc, generated client, validation/error/paging contract test
Transaction	Local TX boundary, idempotency, A→B→C timeout/response-loss/reconcile trace
Integration	HTTP/TCP/message/file provider config, timeout/retry/attempt/DLQ/replay result
Security	identity/permission/data scope/masking/secret/audit negative result
DB	migration/rollback-forward/verify/restore impact
Operations	ADM route, log/metric/trace, alert/runbook, rollback/reconcile owner